USER GUIDE

# PrimeOffice

Pixel office for your AI coding agents

PrimeOffice is a **pixel office for your AI coding agents**. Every Claude Code session on this PC (CLI, Desktop app or IDE) that was active recently becomes a character: it walks to a desk, **types** while it edits files or runs commands, **reads** while it searches, and raises a **bubble** when it needs you. Around the office: **Projects** (check a project folder against the standard and list everything missing), **History** (the latest sessions: last prompt, what was just done, last reply) and **Agents** (which agents are on this PC).

**Version:** 0.7.0 · **Generated:** 2026-11-13

The same content as the in-app Help (press F1).

# Contents


1 **Getting started**
What PrimeOffice is, how it finds agent sessions, what it reads and writes, the layout, dropping folders, shortcuts, appearance and updates.

2 **Office — the pixel office**
Every active agent session is a character: it types while editing, reads while searching, raises a bubble when it needs you. Filter by project and time, pick the crew, click a character for details.

3 **Office — layout editor**
The office's Layout button: place furniture, paint floors, walls and carpets, draw folder areas, erase tiles; Undo / Redo / Save / Reset. The layout saves itself to layout.json.

4 **Agent details**
The details of one session (Office and History): status, model, context, last prompt, what is running, recent actions, last reply.

5 **Chat — give an agent work**
Write what you want done and Claude Code does it in the folder you pick. Every conversation is an ordinary Claude Code session: continue it from a terminal or Claude Desktop.

6 **Projects — project check**
Open a folder, check it against the project standard (agent sessions, CLAUDE.md, git, docs/SPEC.md, docs/PROGRESS.md…) and see how to fix every missing item.

7 **History — recent sessions**
The latest sessions of every project or one: title, status, last prompt, last three actions; click one for the full view.

8 **Agents — providers on this PC**
Claude Code, Codex CLI (ChatGPT), Gemini CLI and DeepSeek: found on this PC or not, where their sessions are, whether the office can show them.

9 **Terminal — the agent's CLI inside the app**
A real terminal running Claude Code, Codex, Gemini or the shell in a project folder — for what needs the CLI's own screen: signing in, /resume, slash commands, answering permission prompts.

10 **Skill library — skills by template**
Each template (Game, Video…) is a set of skills. A Boss added with a template loads that set only; its sub-agents share it, other templates are not loaded.

11 **Command line**
primeoffice.exe check / history / providers: check a project, read its history, detect agents from a terminal or CI, with a JSON report and exit codes.

# 1 Getting started

What PrimeOffice is, how it finds agent sessions, what it reads and writes, the layout, dropping folders, shortcuts, appearance and updates.

## What is PrimeOffice?

PrimeOffice is a **pixel office for your AI coding agents**. Every Claude Code session on this PC (CLI, Desktop app or IDE) that was active recently becomes a character: it walks to a desk, **types** while it edits files or runs commands, **reads** while it searches, and raises a **bubble** when it needs you. Around the office: **Projects** (check a project folder against the standard and list everything missing), **History** (the latest sessions: last prompt, what was just done, last reply) and **Agents** (which agents are on this PC).

- The app's interface is in English; this guide shows button and option names in **bold** exactly as on screen.
- Open this guide any time with `F1` (it opens on the tab you are in, even while the office has focus), the **?** button in the header, or Ctrl+K → **Help / Hướng dẫn**.

## How PrimeOffice finds sessions

Claude Code writes each session as a transcript `<session>.jsonl` in `%USERPROFILE%\.claude\projects\<folder>\`, where `<folder>` is the session's working directory with every character other than letters, digits and `-` turned into `-` (for example `D:\Game` → `D--Game`). PrimeOffice only **reads** these files.

- A background observer checks **every second** for transcripts that changed within the **Active in** window (30 minutes by default) and reads only the new lines. When a session stays quiet longer than that, its character leaves the office.
- The office only observes: no hooks, no change to `~/.claude/settings.json`, nothing is sent over the network. Only the **Chat** tab starts an agent, and only when you send a message.
- Codex CLI sessions (`~/.codex/sessions`) are read like Claude's and join the office. Gemini (`~/.gemini/tmp`) is only checked for existence today — it does not join the office yet (see the **Agents** topic).
- **Waiting for permission** is a **guess** from the transcript: a tool that has not finished and has been quiet for over 10 seconds. Long-running commands (Bash, PowerShell), subtasks and web lookups are never guessed as waiting.

## What PrimeOffice reads and writes

- **Reads**: Claude Code transcripts (above); in a project folder it only checks **which files exist** (CLAUDE.md, .git, docs/SPEC.md…), without reading them.
- It **never writes** into a project folder or into an agent's own files.
- **Writes** to `%LOCALAPPDATA%\com.phineas.primeoffice\`: `projects.json` (the projects you opened), `layout.json` (the office layout saved by the editor) and `office.json` (the office's Settings, agents' seats, folder ↔ area mapping).

- UI preferences (office filter, crew, appearance, help language) are remembered in the app window's storage. The command line's `--report` writes a JSON file exactly where you tell it.

## Layout

- **Navigation rail (left)**: the 6 tabs, in the order of Ctrl+1 … Ctrl+6 — **Office**, **Chat**, **Projects**, **History**, **Agents**, **Terminal**. At the bottom: **Settings** (Ctrl+,), **About & contact**, **Command palette** (Ctrl+K) and **Show log / Hide log** (with an error count while the log is closed).
- **Header**: `PrimeOffice / <tab>`, the **Search actions…** box (opens the command palette), the **?** button (this guide), the version pill ( `browser preview` outside the desktop app — no real data is read there) and `via Abyss` when Abyss launched it.
- **Log (bottom)**: messages from the UI and the Rust side (projects checked, layout saved, errors…). Toggle it with Ctrl+`; Ctrl+K → **Clear log** empties it.

## Opening project folders

**Drag & drop** a folder anywhere on the window, or use **Open folder…** / Ctrl+O / Ctrl+K → **Open project folder…**. The folder is added to the **Projects** list and checked, then, depending on the tab you are in:

- **Office**: if the project is **Ready**, the office filters to it; if not, you are taken to **Projects** to see what is missing.
- **Projects**: the project is selected and its report shown.
- **History**: History filters to that project.
- **Agents**: you are taken to **Projects** to see that project's report.
- Only the first item is used when you drop several. Dropping a file (not a folder) fails the **Folder exists and can be read** check.

## Shortcuts

| | |
|---|---|
| `F1` | Open / close this guide (on the current tab's topic) |
| `Ctrl` `K` | Command palette: tabs, open a folder, run again, updates, log, settings… |
| `Ctrl` `1` … `Ctrl` `6` | Switch tab (Office, Chat, Projects, History, Agents, Terminal) |
| `Ctrl` `O` | Open a project folder (as if dropped on the current tab) |
| `F5` | Run the current tab again: Office refreshes the agent list, Projects re-checks the selected project, History reloads, Agents detects again. Never writes anything |
| `Ctrl` `` ` `` | Show / hide the log |
| `Ctrl` `,` | Settings (appearance) |
| `Esc` | Close a dialog / the command palette / this guide |

- Right after you click in the office, keys go to the office: only `F1` is passed on. Click the header or the list on the right before using Ctrl+K, Ctrl+1…6 or F5. The layout editor's keys are in the **Office — layout editor** topic.

## Appearance and updates

- **Settings** (Ctrl+,): light / dark / Windows theme, UI and code fonts, UI scale, effects level. It does not change the pixel office (the office has its own **Settings**, see the **Office** topic).
- Ctrl+K → **Check for updates…** (or in **About & contact**). Release builds also check once at startup. When a new version exists, the header shows **Update to vX.Y.Z** to download, install and restart.
- When Abyss launches PrimeOffice, it does not check by itself: Abyss shows the update and installs it when the app closes.

# 2 Office — the pixel office

Every active agent session is a character: it types while editing, reads while searching, raises a bubble when it needs you. Filter by project and time, pick the crew, click a character for details.

## Quick start

1. Use Claude Code as usual (terminal, Desktop app or IDE). Within a second of its transcript changing, the session's character walks into the office.
2. Read the label above the character: what it is doing (`Editing App.tsx`, `Running: npm test`, `Searching code`…) and the folder name.
3. Click a character (or its card in the strip under the office) to open its details in the sidebar on the right — see the **Agent details** topic.

## Layout: office, sidebar, agent strip

- **Agent strip** (under the office): one card per agent — state (coloured dot), name, what it is doing, folder, CLI and a **context** bar. Click a card to select it, again to deselect. A dashed card is a terminal opened from the app whose session has no character yet.
- **Sidebar** (right): the selected agent, or the **Command Center** when none is. Drag the divider to size it (double-click: default width); the button at the end of the toolbar hides / shows it.

## Add agent

1. Click **Add agent** at the end of the agent strip.
2. **Identity**: the name on its card, in the Command Center and on its terminal tab (empty: automatic).
3. **Workspace**: the folder it works in (the open studio's folders first) and — for Claude Code — continue an existing session or **New session**.
4. **Engine**: the CLI (**Claude Code**, **Codex**, **Gemini**, **Shell**; greyed = not installed), **Model**, and for Claude Code **Permissions**. The command that will run is shown at the bottom of the dialog.
5. **Briefing**: pick a **Template** or write the **First task** — it is typed into the CLI once the CLI has started, then Enter, as if you typed it.
6. **Start agent**: the CLI runs in a real terminal, shown in the sidebar right away. A new Claude Code session gets its id up front, so when its character walks in, the terminal card merges into that agent's card.

## Command Center

- **Monitor**: every agent — who **needs you** first, then who works; what it is doing, action count, CLI, model, folder and **ctx** (context). Click one to open it.
- **Dispatch**: give work. **To** an agent with a terminal in the app (typed into its terminal, then Enter) or **A new agent…** (pick the CLI and folder; the task becomes its first task). **Ctrl+Enter** sends.
- **Activity**: the latest tool calls of every agent, newest first.

- An agent started outside PrimeOffice has no terminal in the app: open its session in one from its details (Claude Code), or continue it in Chat.

## Top toolbar

- **Project**: **All projects** or one project. Only projects that are **Ready** in the Projects tab are listed.
- **Open folder...**: opens a project folder; if it is Ready the office filters to it, otherwise you go to Projects.
- **Active in**: `15 min`, `30 min`, `1 h`, `4 h`, `24 h` — sessions whose transcript changed within this window are in the office; quieter ones leave. The same window decides the **Idle** status in History.
- **Crew**: **PrimeOffice crew** (8 characters drawn by the app's own generator) or **Classic (pixel-agents)**. Switching crew reloads the office.
- The **N agents** pill counts the characters; the last button hides / shows the sidebar. Project, Active in, Crew and the sidebar width are remembered for next time.

## Characters, labels and bubbles

- **Typing**: editing / writing files, running commands, other tools. **Reading**: `Read`, `Grep`, `Glob`, `WebFetch`, `WebSearch`.
- **Green check**: the turn is done; the agent waits for your next prompt.
- **Waiting for input** label: the agent asked you something (`AskUserQuestion`) or waits for plan approval (`ExitPlanMode`).
- **...** bubble: most likely waiting for your permission (a guess: a tool quiet for more than 10 seconds). Check the Claude Code window to be sure.
- The small bar under the label is the **context** used; hover it for the token count.
- Subtasks (`Task` / `Agent`) show as a `Subtask: …` label on the agent itself, not as separate characters yet.

## Moving around the office

- Click a character to select it: the camera follows it and its details open on the right. Click it again or click empty floor to deselect.
- With a character selected, click a **free seat** to move it there; click its own seat to send it back. Seats are remembered per session, across crew switches and app restarts.
- Mouse wheel / two-finger scroll: pan. Ctrl+wheel or the **+ / −** buttons at the top left: zoom. Middle-button drag: pan.
- The column of small cards on the office's right edge: one card per agent (portrait + status dot); click to select and follow.
- The × (**Close agent**) button sends the character out of the office but **does not stop** the agent (PrimeOffice only observes): it walks back in, to the same seat, as soon as its session does something new. A character also leaves by itself once its session is quiet for longer than **Active in**.

## The office's bottom bar

- **Layout**: turns on the layout editor — see the **Office — layout editor** topic.

- **Settings** (the office's own): **Sound Notifications** (a chime when an agent finishes / needs you), **Always Show Labels** (labels above every character), **Show Areas** (only when Areas are available), **Debug View** (a raw status list instead of the office).
- **Sound Notifications**, **Always Show Labels** and **Show Areas** are saved ( `office.json` ) and survive crew switches and app restarts. Defaults: sound off, labels always shown, areas hidden.

## Studios — one company per tab

- The tab strip on top of Office holds the **studios**. Each studio is a company of its own: its floor, its departments and **one project**. Click a tab to switch; the office reloads that studio's floor.
- + creates a studio: pick its **project** (a folder with its own `.claude` — a folder without `.claude`, or already another studio's project, cannot be picked), its **kind** (**Game studio** with the Campus or Open Floor theme, **Video studio**, **Classic office**) and a name. At most 12 studios.
- **Studio…** (right of the strip): rename it, see or change its **project** (only agents working in that project appear here), or delete the studio and its floor. A studio made before 0.4 has no project yet and shows every project until you choose one. A folder must be opened first with **Open folder…** or the Projects tab.
- In a Video studio sub-agents sit by type: `scriptwriter` → Script room, `video-editor` → Editing bay, `thumbnail-designer` → Motion & graphics, `voice-over-artist` → Audio booth, `seo-specialist` → Publishing & SEO, `retention-analyst` → Analytics, `showrunner` → Ban sản xuất. These names are placeholders until the studio's skill set is chosen.
- The studio list is saved in `studios.json` ; each studio's floor in `layout.json` (the first studio) and `layout-<id>.json` .

## Theme Studio, rooms and departments

- **Theme Studio…** (top bar) changes the floor of the studio on screen. **Game studio** has two themes: **Campus** (a room per department with walls and doors, Pantry right under Studio head) and **Open Floor** (an executive wing with walls: Ban giám đốc, Phòng họp, Studio head, Loc & community; the departments share one open floor, set apart by areas and floor colours). **Video studio** and **Classic office** are as before. Loading a theme replaces the whole layout.
- **Special themes** (Game studio, in Theme Studio): **Lễ hội** — Christmas, Halloween, Tết; **Phong cách** — Cyberpunk, Matrix, Aquarium; **DC** — **Hall of Justice** and **BatCave**. Each keeps the departments, seats, Pantry and Ban giám đốc of Campus or Open Floor, with its own floor colours, decor and an effect over the floor (snow, petals, code rain…). **Effects** in the top bar turns the effect on or off.
- **Avatar**: Add agent → Identity → Avatar picks how the agent looks — Auto (a crew character, as before), any character of the project, or a themed one: DC, Marvel, Dragon Ball, Naruto, Lễ hội, Cyber & Sea. Change it later with the avatar button on the agent's card. Sub-agents keep the crew look.
- **Add a room** (in Theme Studio): pick one of 12 ready rooms (small / medium / large department, Ban giám đốc, Phòng họp, Pantry, Studio head, Lab / Server, Art, Audio, QA, empty), name it and click Add room: it is built in the first free spot with a way out to the floor — walls, a doorway, furniture and an area with that name (name it after a department, e.g. `Engineering 2` , `Phòng QA` , so its sub-agents sit there). When the floor is full it grows (up to 64 × 64 tiles). Then arrange desks, chairs and PCs tile by tile with the office's **Layout** button.

- **Keep a room as a template**: pick an area of the floor, name it, click Keep — that room (size, floor, furniture, wall decor) becomes a room of your own, usable in every studio; delete it with the bin on its card. Save the layout in Layout before keeping it.
- Every sub-agent ( `Agent` / `Task` ) is its own character: it appears beside the session that started it, then **walks to a desk in its department** by agent type ( `gameplay-programmer` → Engineering, `ue-gas-specialist` → Engine lab, `qa-tester` → QA …). Its label: agent type · task; select it to see the tool it is running.
- Main sessions sit at **Studio head**; when their **turn is done** they rest in the **Pantry** (Video studio: the Lounge) and come back to their desk for new work or a question for you. Agents without a department ( `general-purpose` , `Explore` ) stand by the session that started them.
- **Directors work only in Ban giám đốc**: the **Boss** of a folder is its director — it sits in Ban giám đốc and waits there between turns; director sub-agents (creative-director, technical-director, producer) sit there too.
- **Name and Master Task**: every session has a **person's name** (Batman, Robin…) — the one given in Add agent, the one in its first message "You are …", or the next free one the Office gives it (rename with the pencil). The session's title is its **Master Task**: the short title Claude Desktop shows for it, else what its first message asks for (without the role text).
- **Status over a character**: the running tool (Editing…, Running…), **Thinking…** while it thinks or writes, **Needs approval**, **Waiting for your answer**, **✓ Done** when its turn is over, **Waiting on sub-agents 1/2** when its turn is over but sub-agents still work, **✓ All done** once every sub-agent has finished; **Idle** only when quiet longer than Active in.
- **Add sub-agent** (the Boss's card): the Boss hands a task to a sub-agent — pick its type (general-purpose, Explore, Plan, the agents in `.claude/agents` ), a title and the task. The order is typed into the Boss's terminal (opened with `--resume` when it has none); for a session open in Claude Desktop press **Copy** and paste it there. The Boss's card lists its sub-agents, ticking each when it finishes; a finished sub-agent shows a green ✓ for a few seconds, then leaves the office.
- **Who is the Boss**: the first agent added to a folder with no Boss is its Boss (Add agent picks it). A **new session** (from Claude Code, Claude Desktop, a terminal…) in a folder with no other agent at work becomes its Boss too; when the Boss leaves the office, the first agent still at work there takes the crown. The role is in the session's first message, so PrimeOffice reads it back from the transcript: a Boss opened in a terminal or in Claude Desktop keeps its crown, and a "ghost" Boss (closed before its first message) is dropped — the next agent becomes the Boss. A session's folder is where it started; a later `cd` does not change it.
- **Phòng họp — handing work down**: when a sub-agent starts, the one handing it the work and the sub-agent meet in the meeting room for a few seconds, then each goes back to its own room. The one handing it down is the lead of its department when at work (lead-programmer briefs gameplay-programmer, qa-lead briefs qa-tester …), else a director (a director sub-agent, or the Boss). A full meeting room skips the briefing. **Editor desk** (in Engine lab): an agent calling the editor through MCP ( `mcp__unreal-mcp__call_tool` …) sits there, **one at a time** like Unreal MCP; the next one waits at its own desk and takes the desk when it is free. A department with no free desk: the agent stands in the room.
- Departments are found by **Area name**, ignoring case and accents: an Area named `Engineering` , `QA` , `Phòng QA` , `Engine lab` , `Unreal lab` … in your own layout works too. An Area named after one agent (e.g. `writer` ) is that agent's own desk.

## The right column

- Each row: the session title (or folder name), status, what it is doing or did last, folder · time. The list refreshes every 2 seconds; F5 refreshes it at once.
- Statuses: **Working** (running), **Needs you** (asking / waiting for permission), **Turn done** (turn finished), **Idle** (quiet longer than Active in).
- Click a row for its details; **Back to the list** returns.

## Empty office?

- **The office is quiet** means no session changed within **Active in** (and in the selected project). Try a longer window or **All projects**.
- Open the **Agents** tab: Claude Code should be **Active** or **Found**. **Not found** means `~/.claude/projects` does not exist yet — run `claude` once.
- The project filter matches the session's exact working directory: sessions started in a parent or a subfolder do not count.

# 3 Office — layout editor

The office's Layout button: place furniture, paint floors, walls and carpets, draw folder areas, erase tiles; Undo / Redo / Save / Reset. The layout saves itself to layout.json.

## Turning the editor on / off

1. Click **Layout** in the office's bottom bar (it lights up while editing).
2. Pick a tool in the button row: **Furniture**, **Floor**, **Walls**, **Pets**, **Areas** (when available), **Erase**. Click the active tool again to go back to selecting.
3. Click **Layout** again (or press `Esc` a few times) to leave.

## Tools

- **Furniture**: pick a group (**Desks**, **Chairs**, **Storage**, **Tech**, **Decor**, **Wall**, **Misc**), pick an item, click the floor to place it. **Copy** takes the type of an item already placed. The colour for new items is set in the colour panel above.
- Select a placed item (click it): drag to move it (hold Alt as you start dragging to drag out a copy), use the rotate / delete buttons next to it, and the colour panel to recolour it (**Reset** restores the original).
- **Carpet** (inside Furniture): paint carpets; pick a carpet style, colour and **Accent** colour; **Copy** ( `P` ) takes style + colours from an existing tile.
- **Floor**: pick a floor pattern, then click / drag to paint; **Color** opens colour and **Contrast**; **Copy** takes pattern + colour from an existing tile.
- **Walls**: click / drag to add or remove walls; pick a wall set and colour.
- **Erase**: turns tiles into void (outside the room). In Floor, Walls, Carpet, Areas and Erase, the **right mouse button** erases too.
- **Pets**: PrimeOffice ships no pets yet, so this panel is empty.

## Folder areas (Areas)

**Areas** reserve part of the office for a folder: that folder's agents sit inside it. The **Areas** button only shows when the layout already has areas or the office has seen at least one agent (its folder can be mapped).

1. Type a name in **Area name...**, pick a colour, click **Add**.
2. Select the area's card, then paint its tiles on the map (right-click removes).
3. In the card, **Map a folder...** assigns a folder (an agent's folder name); double-click the name to rename it, use the remove button to delete the area.

- Areas are saved with the layout; the folder ↔ area mapping is saved on its own ( `office.json` ), so both survive crew switches and app restarts.

## Saving, Undo, Reset

- Every change is **saved automatically** about half a second later to `%LOCALAPPDATA%\com.phineas.primeoffice\layout.json` (the log says `Layout saved`). Without that file the office uses the bundled default layout.
- Once something changed, the top bar shows **Undo** (Ctrl+Z), **Redo** (Ctrl+Y / Ctrl+Shift+Z), **Save** and **Reset**.
- **Save** marks the current layout as the checkpoint; **Reset** → **Yes** returns to it (the last Save, or the layout the office opened with).
- PrimeOffice has no layout export / import yet; copy `layout.json` to back it up.

## Editor keys

| | |
|---|---|
| `R` | Rotate the selected item |
| `T` | Toggle the selected item's state (e.g. on / off) |
| `Delete` / `Backspace` | Delete the selected item |
| `Ctrl` `Z` | Undo |
| `Ctrl` `Y` / `Ctrl` `Shift` `Z` | Redo |
| `P` | In Carpet: copy a carpet from an existing tile |
| `Esc` | Steps back: drop the item in hand → close the tool → deselect → leave the editor |

- These keys only work while the office has focus (click in the office first).

# 4 Agent details

The details of one session (Office and History): status, model, context, last prompt, what is running, recent actions, last reply.

## Where it opens

- **Office**: click a character or a row in the right column (last 15 actions).
- **History**: pick a session in the left list (last 50 actions).

## Reading the panel

- **Title**: the title set in Claude Code (custom title / agent name) or its summary; otherwise the start of the first prompt; `Untitled session` when empty.
- Info line: agent ( `Claude Code` ), where it runs ( `cli` , `Desktop app` ...), model, last activity, prompt count; below it, the working folder.
- **Context N%**: context used (out of 200k tokens, or 1M for `[1m]` models); green below 60%, amber up to 85%, red above. Hover for the token count.
- **Last prompt**: your latest prompt (commands such as `/init` show as the command), up to 500 characters.
- **Running now**: tools that started and have not finished.
- **Recent actions**: newest first, with the time: reads, edits, commands, searches, web, subtasks, questions, planning, skills...
- **Last reply**: the agent's last text answer, up to 500 characters.

## How the status is worked out

- **Working**: the turn is running.
- **Needs you**: it asked you / waits for plan approval, or a tool has been quiet for over 10 seconds (guessed as a permission prompt).
- **Turn done**: the turn ended, or the last record is a text reply and it has been quiet for over 5 seconds.
- **Idle**: the transcript has not changed within the Office's **Active in** window.

## Tips

- You can select and copy the **Last prompt** / **Last reply** text.
- The summary is mechanical (from tools and prompts); no LLM is called.
- Very long transcripts are read head (256 KB) and tail (4 MB) only, so the prompt count can be partial.

# 5 Chat — give an agent work

Write what you want done and Claude Code does it in the folder you pick. Every conversation is an ordinary Claude Code session: continue it from a terminal or Claude Desktop.

## How to use it

- Pick a **Folder** (a project folder — the open studio's first), write the task, **Enter** to send (**Shift+Enter** for a new line). The reply streams in; each tool the agent uses is a line (running, done, failed).
- The left column lists the **folder's Claude Code sessions** — including the ones you started in a terminal or Claude Desktop. Click one to **continue** it; **New session** starts fresh. Continuing reads the transcript and shows the **whole conversation** (prompts, replies, tools); a very long one shows its end.
- **Stop** ends the running turn. A turn keeps running while you are on another tab; the session's character appears in Office like any session.
- Do not message the **same session** from two places at once (Chat and Claude Desktop): they would write over each other.

## What the agent may do (May)

- A Chat turn cannot stop to ask you: what is not allowed is **refused**, with a note at the end of the turn.
- **Edit files** (default): reads and edits files in the folder; commands run only when the project's Claude Code settings allow them. **Plan only**: reads and plans. **Ask first**: only what your settings already allow. **Full auto**: no permission checks — only in a folder you can afford to lose.

## Your account and Claude Desktop

- PrimeOffice runs **Anthropic's own `claude` CLI** on your PC (`claude -p`, its documented non-interactive mode). The CLI signs in by itself as in a terminal; PrimeOffice reads no token and calls no Anthropic server. Chat turns count against your plan's limits.
- No CLI yet: install Claude Code, run `claude` once in a terminal and sign in there.
- A session made in Chat is a real Claude Code session (`~/.claude/projects`). In **Claude Desktop** type `/resume` in the prompt box to open it; it does **not appear by itself** in Desktop's sidebar, because Desktop lists only the sessions it created. In a terminal: `claude --resume`.
- Chat runs Claude Code only for now. Codex and Gemini come later.

# 6 Projects — project check

Open a folder, check it against the project standard (agent sessions, CLAUDE.md, git, docs/SPEC.md, docs/PROGRESS.md…) and see how to fix every missing item.

## How to use it

1. Click **Open folder…** (or Ctrl+O, or drop a folder on the window).
2. The project goes to the top of the left list — green dot **Ready**, red dot **not ready** — and its report shows on the right.
3. Fix the failed items (each has a **Fix:** line with a copy button), then click the refresh button (**Check again**, F5).
4. Once it is **Ready**: **View in office** shows only this project's agents; **History** opens its history.

## The report

- The **Ready** or **Not ready — N required missing** pill, the path, session counts per agent and the last one.
- The coloured banner lists what is missing at a glance; **Project standard — X/Y passed** lists every item, failed ones first, with its code (`agent.sessions`…).
- Levels: **required** (missing means it cannot be viewed yet), **recommended**, **optional**.
- The bin button **Remove from the list** only removes it from the list — the folder is not touched. The folder button shows it in Explorer.

## Checks and how to fix them

- **Folder exists and can be read** (required, `folder.exists`): the folder was moved, renamed or is on a missing drive. Open it again and remove the old entry.
- **Agent sessions for this folder** (required, `agent.sessions`): no agent has run in **this exact** folder yet. Open a terminal there and run `claude` once (or open this folder in Claude Desktop / your IDE), then check again. Sessions started in a parent folder do not count.
- **Agent instructions (CLAUDE.md or AGENTS.md)** (recommended, `agent.instructions`): run `/init` in Claude Code in this folder, or write `CLAUDE.md` yourself. `AGENTS.md` and `.claude/CLAUDE.md` count too.
- **Git repository** (recommended, `vcs.git`): run `git init` in the folder.
- **Spec (docs/SPEC.md)** (recommended, `abyss.spec`): write `docs/SPEC.md` — goals, main flows, architecture (per Abyss `docs/app-specs/00_SHARED_FRAMEWORK.md` §3).
- **Progress log (docs/PROGRESS.md)** (recommended, `abyss.progress`): add `docs/PROGRESS.md` — status, phases, decisions (template in §15 of the shared framework).
- **Project agent settings (.claude/)** (optional, `agent.settings`): create `.claude/` if the project needs its own settings, skills or agents.
- **Changelog** (optional, `abyss.changelog`): add `docs/CHANGELOG.md` (or `CHANGELOG.md` at the root).
- **README.md** (optional, `abyss.readme`): add a `README.md` saying what the project is and how to run it.

## Notes

- PrimeOffice **never creates** these files for you: it only checks whether they exist. The project list is kept in `%LOCALAPPDATA%\com.phineas.primeoffice\projects.json`.
- Every listed project is re-checked whenever the tab opens. The line under each name in the left list counts what is missing by level, e.g. `2 recommended, 1 optional missing` (or `nothing missing`); a project that is not Ready says `not ready`.
- The same check runs in CI: `primeoffice check <folder>` (see the **Command line** topic).

# 7 History — recent sessions

The latest sessions of every project or one: title, status, last prompt, last three actions; click one for the full view.

## How to use it

1. Pick a **Project**: **All projects** (every Claude Code session on this PC) or a project opened in Projects.
2. Pick **Show**: `10`, `30`, `60` or `120 latest` — how many of the newest sessions (by when the transcript last changed).
3. Click a session on the left to see its details on the right (see the **Agent details** topic). **Refresh** (F5) reloads.

## What each row shows

- The title (or `Untitled session`) and status (**Working**, **Needs you**, **Turn done**, **Idle**).
- The last prompt (2 lines), the three latest actions with an icon per kind, then folder · time.

## Tips

- Drop a folder on the History tab to open it as a project and filter History to it at once.
- **Idle** uses the Office's **Active in** window: sessions older than that are Idle.
- Claude Code and Codex CLI sessions appear here; Gemini comes later. History only reads transcripts; it never deletes or edits anything.
- The Project / Show choice is not remembered when the app restarts.

# 8 Agents — providers on this PC

Claude Code, Codex CLI (ChatGPT), Gemini CLI and DeepSeek: found on this PC or not, where their sessions are, whether the office can show them.

## What each card shows

- State: **Active** (sessions changed in the last 30 minutes), **Found** (its session folder exists), **Not found**, **Planned** (known but not supported yet).
- A note, the session folder path, recent sessions (30 minutes) and **Shown in the office** / **Not in the office yet**.
- **Refresh** (or F5) detects again.

## Each agent

- **Claude Code**: fully supported — `~/.claude/projects`, CLI, Desktop app and IDE sessions. **Not found**: run `claude` once to create the folder.
- **Codex CLI (ChatGPT)**: reads the rollout files in `~/.codex/sessions` — prompts, commands, edited files, the last reply, context used. Waiting for permission is not guessed yet. **Not found**: run `codex` once.
- **Gemini CLI**: only `~/.gemini/tmp` is checked; reading its sessions comes in a later version.
- **DeepSeek**: always **Planned** — no local session format yet; it will come through the tool that runs DeepSeek.

- This tab only reads; it never installs, configures or starts an agent. Dropping a folder on this tab (or Ctrl+O) opens it as a project and takes you to **Projects**.

# 9 Terminal — the agent's CLI inside the app

A real terminal running Claude Code, Codex, Gemini or the shell in a project folder — for what needs the CLI's own screen: signing in, /resume, slash commands, answering permission prompts.

## How to use it

- Pick a **Folder**, then click a button under **New terminal**: **Claude Code**, **Codex**, **Gemini** or **Shell** (PowerShell). A greyed button means that CLI is not installed — hover it for the install command.
- Each terminal is a tab; up to 8 at once. Green dot: running; **ended**: the program has exited (its screen stays readable). × ends the terminal and closes its tab.
- A terminal keeps running while you are on another tab. Closing PrimeOffice ends every terminal.
- In **Chat**, the **Terminal** button opens Claude Code in the chosen folder — on the session on screen (`claude --resume`) when there is one. Do not message the same session from Chat and Terminal at once.
- A session run in Terminal is an ordinary session of that CLI: its character appears in **Office** and it is listed in **History**.

## Keys

| | |
|---|---|
| `Ctrl` `C` | Copies when text is selected; otherwise sends Ctrl+C to the program |
| `Ctrl` `V` | Paste |
| `Ctrl` `1` … `Ctrl` `6` | Still switch the app's tabs |

- While the cursor is in a terminal, the other Ctrl keys (Ctrl+K, Ctrl+O, Ctrl+`…) belong to the running program; they do not open the app's palette or log.

## Your account

- Terminal runs **the vendor's own CLI** like a terminal window: you type, the CLI answers. Sign in with the CLI's own flow (`/login` in Claude Code). PrimeOffice only passes keys and text through — it reads no token and does not type for you.

# 10 Skill library — skills by template

Each template (Game, Video…) is a set of skills. A Boss added with a template loads that set only; its sub-agents share it, other templates are not loaded.

## How to use it

- **Add agent → Engine → Skill template**: pick the Boss's template. The one named like the studio's kind ( `game` , `video` ) is preselected. Members use their Boss's; a resumed session keeps the one it started with.
- **Settings (Ctrl+,) → Skill library**: see the folder, move it (the structure is created, existing files are kept), and see each template's skills and warnings.
- **Only this template's skills** (on by default): adds `--setting-sources project,local` , so personal skills/plugins in `~/.claude` and the claude.ai account's skills are not loaded (nor the session's personal settings). Claude Code's built-in skills ( `code-review` , `init` …) remain.

## Folder structure

- `templates/<id>/skills/<name>/SKILL.md` — your own skills; `agents/` — your agents; `repos/` — third-party clones; `TEMPLATE.md` — name and description. Full guide: `README.md` in the folder.
- Repos in `repos/` : plugin ( `skills/` or `.claude-plugin/` ) → `--plugin-dir` ; marketplace ( `plugins/*` ) → each plugin; project ( `.claude/skills` or `.claude/agents` ) → `--add-dir` , its skills and agents.
- Skills are attached when a session starts: after changing the template, close the Boss's terminal and open it again.

# 11 Command line

primeoffice.exe check / history / providers: check a project, read its history, detect agents from a terminal or CI, with a JSON report and exit codes.

## Commands

| | |
|---|---|
| `check <thư mục>` | Check against the project standard, print every missing item with its fix |
| `history <thư mục> --last N` | The N newest sessions (default 5): last prompt, recent actions, last reply |
| `providers` | Claude Code / Codex / Gemini / DeepSeek on this PC |
| `--help` | Help and examples |

- `check` : `--report r.json` writes the JSON report (the same structure the app shows), `--fail-on warn|error` (default `error`), `--quiet` prints only the summary and errors. `history` : `--report r.json`.
- Exit codes: `0` done, no issue at the `--fail-on` level; `1` issues at that level (e.g. a project that is not Ready); `2` bad arguments or a missing folder.
- In `cmd.exe` use `start /wait primeoffice.exe …` to wait for the exit code; PowerShell and CI call it directly. Any other first argument opens the app window as usual.
- `primeoffice.exe <folder>` (or **Open with** PrimeOffice on a folder in Explorer): the window opens and the folder is opened as if dropped on the **Office** tab — the office filters to it when it is Ready, otherwise you are taken to **Projects**. An argument that is not a folder is ignored (with a warning in the log).